02 CONTROLLERS IN NESTJS — THEORY

A Controller is a class responsible for handling incoming requests and returning a response to the client. It is the "front door" of your API — it decides which HTTP requests go where, but it does NOT contain business logic itself.

```
Client Request -> Controller -> Service -> (Database/other logic)
                  -> Controller -> Response
```

1. DECLARING A CONTROLLER

```ts
import { Controller, Get } from '@nestjs/common';

@Controller('cats')
export class CatsController {
  @Get()
  findAll(): string {
    return 'This returns all cats';
  }
}
```

- @Controller('cats') tells Nest: "every route in this class starts with /cats".
- @Get() with no argument means this method handles GET /cats.
- Whatever the method returns is automatically serialized (to JSON by default) and sent as the response body — no manual res.send() needed.

2. REGISTERING IT

A controller does nothing on its own — it must be listed in a module:

```ts
import { Module } from '@nestjs/common';
import { CatsController } from './cats.controller';
import { CatsService } from './cats.service';

@Module({
  controllers: [CatsController],
  providers: [CatsService],
})
export class CatsModule {}
```

Nest reads this at startup and registers the routes with the underlying HTTP server (Express or Fastify).

3. ROUTE PARAMETERS, QUERY, AND BODY

```ts
import { Controller, Get, Post, Body, Param, Query } from '@nestjs/common';

@Controller('cats')
export class CatsController {
```

```
  constructor(private readonly catsService: CatsService) {}

  @Get()
  findAll(@Query('breed') breed?: string) {
    // GET /cats?breed=persian
    return this.catsService.findAll(breed);
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    // GET /cats/5
    return this.catsService.findOne(id);
  }

  @Post()
  create(@Body() createCatDto: CreateCatDto) {
    // POST /cats  (body: { name, breed, age })
    return this.catsService.create(createCatDto);
  }
}
```

Decorator	Pulls from	Example
@Param('id')	URL path segment	/cats/5 → id = '5'
@Query('breed')	query string	/cats?breed=persian → breed='persian'
@Body()	JSON request body	POST payload → typed DTO object
@Headers('authorization')	request headers	bearer token, etc.

4. FULL CRUD EXAMPLE (THE COMMON SHAPE)

```
@Controller('cats')
export class CatsController {
  constructor(private readonly catsService: CatsService) {}

  @Get()                      // GET    /cats
  findAll() { return this.catsService.findAll(); }

  @Get(':id')                 // GET    /cats/:id
  findOne(@Param('id') id: string) { return this.catsService.findOne(id); }

  @Post()                     // POST   /cats
  create(@Body() dto: CreateCatDto) { return this.catsService.create(dto); }

  @Put(':id')                 // PUT    /cats/:id
  update(@Param('id') id: string, @Body() dto: UpdateCatDto) {
    return this.catsService.update(id, dto);
  }

  @Delete(':id')               // DELETE /cats/:id
  remove(@Param('id') id: string) { return this.catsService.remove(id); }
}
```

This one controller now exposes a complete REST resource for "cats". Note every method is one line — it just forwards to catsService. That's the pattern to copy.

5. WHY THE CONTROLLER STAYS "THIN"

The controller in the example above never touches a database or runs an if statement for business rules. That logic lives in CatsService:

```
@Injectable()
export class CatsService {
  private cats: Cat[] = [];

  findAll(breed?: string) {
    return breed ? this.cats.filter(c => c.breed === breed) : this.cats;
  }

  create(dto: CreateCatDto) {
    const cat = { id: Date.now().toString(), ...dto };
    this.cats.push(cat);
    return cat;
  }
}
```

Nest automatically creates and injects CatsService into CatsController via the constructor — this is Dependency Injection. The controller just asks for it as a constructor parameter; Nest handles instantiation.

Why keep logic out of the controller:

- Testability — you can unit test CatsService without HTTP at all, and test CatsController with a mocked service.
- Reuse — the same CatsService could be called from a CLI command, a scheduled cron job, or a GraphQL resolver — not just this REST route.
- Swappability — you could switch from REST to GraphQL by writing a new resolver that calls the same CatsService, unchanged.

6. ATTACHING CROSS-CUTTING BEHAVIOR

Controllers (or individual routes) can have guards, pipes, and interceptors attached via decorators, instead of writing that logic inline:

```
@UseGuards(AuthGuard)          // blocks the request unless authenticated
@UsePipes(ValidationPipe)      // validates @Body() against the DTO's rules
@Controller('cats')
export class CatsController { ... }
```

This is how auth checks, input validation, logging, and response transformation get layered onto routes without cluttering the controller's own methods.

★ RULE OF THUMB

If you're about to write a database query, a calculation, or an if/else business rule inside a controller method — stop. Move it into a service. The controller's only job is: read the request, call the service, return the result.